

Day 2 - Control Flow: Conditionals, Loops, Truthy/Falsy

Day 2 — Control Flow: Conditionals, Loops, Truthy/Falsy

Objectives

- Use `if/else if/else` to make decisions
- Use `for`, `while`, and the two special JavaScript loops `for...of` and `for...in`, and know when each is right
- Use `switch` as an alternative to a long `if/else if` chain
- Use the ternary operator to pick between two values
- Solidify yesterday's truthy/falsy rules, now that you can actually use them in real conditions

`if` / `else if` / `else`

```
const age = 20;

if (age >= 18) {
  console.log("You can vote.");
} else {
  console.log("You cannot vote yet.");
}
```

Exactly like English: "if `age` is greater than or equal to 18, run the first block; otherwise, run the second." The curly braces `{ }` mark which lines belong to each branch (recall from Day 0: unlike Python, JavaScript uses braces, not indentation, to decide this — indentation here is purely for human readability).

Chain additional conditions with `else if`, checked in order, top to bottom, stopping at the first one that's true:

```
const score = 72;
let grade;

if (score >= 90) {
  grade = "A";
} else if (score >= 80) {
  grade = "B";
} else if (score >= 70) {
  grade = "C";
} else {
  grade = "F";
}

console.log(grade); // "C"
```

Notice `grade` is declared with `let` above the `if`, then assigned inside it — this is a common pattern, since a variable declared with `const` *inside* one branch of an `if` wouldn't exist anymore once that branch's `{ }` block ends (you'll understand exactly why once you cover scope in depth on Day 4).

Comparison and logical operators

`===/!==` (Day 1), `>`, `<`, `>=`, `<=`. Combine conditions with `&&` (and), `||` (or), `!` (not):

```
if (age >= 18 && hasId) {
  console.log("allowed in");
}

if (isWeekend || isHoliday) {
  console.log("no work today");
}

if (!isRaining) {
  console.log("no umbrella needed");
}
```

Unlike Python, JavaScript does **not** support chained comparisons like `0 <= x < 10` the way you might expect — that expression is legal JavaScript, but doesn't do what you'd think (it evaluates `0 <= x` first, producing `true/false`, then compares THAT boolean to `10`, which is almost never what you meant). Always write chained range checks out fully and explicitly:

```
if (x >= 0 && x < 10) { // the CORRECT way to check a range in JavaScript
  console.log("x is between 0 and 9");
}
```

The ternary operator — an `if` that produces a value

```
const status = age >= 18 ? "adult" : "minor";
```

Read as: "the value is 'adult' IF `age >= 18`, otherwise (`:`) the value is 'minor'." Use this only when each branch is a single simple value; use a full `if/else` for anything more involved.

Loops

`for` — the classic counting loop

```
for (let i = 0; i < 5; i++) {
  console.log(i); // prints 0, 1, 2, 3, 4
}
```

A `for` loop has three parts, separated by semicolons, all inside the parentheses: **initialization** (`let i = 0` — runs once, before the loop starts), **condition** (`i < 5` — checked before every single pass; the loop stops the moment this becomes false), and **increment** (`i++` — runs after every pass through the loop body). `i++` is shorthand for `i = i + 1`; you'll also see `i += 1` (add-and-assign) used the same way.

Just like Python's `range()`, this starts at 0 and stops *before* reaching 5 — five iterations total (0, 1, 2, 3, 4), by convention, though unlike Python there's no separate `range` object here; you write the counting logic out explicitly yourself every time.

`while` — repeat until a condition becomes false

```
let count = 0;
while (count < 3) {
  console.log("still going:", count);
  count++;
}
```

```
}  
console.log("done");
```

Use **while** when you don't know in advance exactly how many times you'll repeat something — you're waiting for some condition to change, rather than counting through a known range. **Forgetting to update the loop's condition variable inside the loop body creates an infinite loop** — the program will run forever. If your program seems to hang, this is the first thing to check.

`for...of` — looping over the VALUES in an array (or string)

```
const fruits = ["apple", "banana", "cherry"];  
  
for (const fruit of fruits) {  
  console.log(fruit);    // "apple", then "banana", then "cherry"  
}  
  
for (const character of "abc") {  
  console.log(character); // "a", "b", "c"  
}
```

This is JavaScript's closest equivalent to Python's **for item in some_list:** — walking through each **value** in an array, one at a time, in order. You'll use **for...of** far more often than the classic counting **for** loop above, once you're mainly working with collections of data (arrays, covered fully Day 3) rather than doing pure counting.

`for...in` — looping over the KEYS/property names of an object

```
const scores = { Ana: 90, Bo: 85 };  
  
for (const name in scores) {  
  console.log(name, scores[name]); // "Ana 90", then "Bo 85"  
}
```

for...in gives you each property **name** (a string), not the value directly — you then use square brackets, **scores[name]**, to look up the actual value for that name. **Important, common point of confusion: for...in is for objects; for...of is for arrays (and strings, and other "iterable" collections).** Using **for...in** on an array technically works but is widely considered bad practice (it can include unexpected extra properties and doesn't guarantee numeric order) — always use **for...of** for arrays, and save **for...in** specifically for objects. You'll get much more comfortable with this distinction once you've properly met objects and arrays tomorrow (Day 3).

`break` and `continue`

Identical in spirit to Python's versions:

```
for (const number of [1, 2, 3, 4, 5]) {  
  if (number === 3) {  
    break;          // stop the loop entirely the instant we hit 3  
  }  
  console.log(number); // prints only 1, then 2  
}  
  
for (const number of [1, 2, 3, 4, 5]) {  
  if (number === 3) {  
    continue;       // skip JUST this one pass, keep looping  
  }  
}
```

```
    }  
    console.log(number);    // prints 1, 2, 4, 5 -- skips only 3  
}
```

`switch` — a multi-way branch for comparing one value against several exact possibilities

```
function describeDay(dayNumber) {  
  switch (dayNumber) {  
    case 1:  
      return "Monday";  
    case 2:  
      return "Tuesday";  
    case 6:  
    case 7:  
      return "Weekend!";    // falling through two cases with no break between them -- both hit this line  
    default:  
      return "Unknown day";  
  }  
}
```

`switch` checks `dayNumber` against each `case` value, top to bottom, and runs the code under the first match. `default` is the catch-all, equivalent to a final `else`. Notice `case 6:` has no code directly under it before `case 7:` — this is called "falling through," and it means both `6` and `7` end up running the exact same `return "Weekend!"` line. **A classic, very common `switch` bug**, in situations where you're not returning immediately like this example: forgetting a `break;` statement at the end of a case, which causes execution to keep running into the *next* case's code too, even though you only meant to match one — always double check whether you need an explicit `break;` (not needed here, since `return` already exits the function immediately, but you'll need it in cases that don't return).

Exercises

Open `exercises.js`, fill in each `// TODO`, and run `node exercises.js` to check your work.